

BDC LEARNING PLATFORM · 2026

Recommender Systems & Conversational RS

From fundamental theory to production deployment — an end-to-end guide for
the BDC CoreApplication project

● 25 Slides ● RS Fundamentals ● Algorithms & Modeling ● Production Deploy ● BDC Integration

OVERVIEW

Presentation Agenda

**Part 1 · RS Intro**

Definition, motivation, business impact, and CRS paradigms. Slides 3–6

**Part 2 · Modeling**

Data representations, cold starts, feature engineering, and BKT. Slides 7–10

**Part 3 · Algorithms**

Collaborative Filtering, sequential models, RL tuning, and Spaced Repetition. Slides 11–15

**Part 4 · Evaluation**

Offline metrics, online A/B testing, and temporal splitting. Slides 16–18

**Part 5 · Deploy**

Production pipelines, serving SLAs, and MLOps feedback loops. Slides 19–21

**Part 6 · BDC System**

CoreApplication architectural implementation & DA workflows. Slides 22–25

PART 1 · INTRODUCTION

What is a Recommender System?

Core Definition

A software engine that automatically predicts **user preference** and presents a personalized feed of items **most relevant** to their interests, drawing from historical actions and content attributes.



Netflix

80% of watched content comes from recommendation cards. Saves \$1B+/year.



Amazon

35% of total sales originate from cross-sell widgets ("Frequently bought together").



Spotify Discover Weekly

Generates over 2.3 billion personalized streams every single week.

MATHEMATICAL FORMULATION

$$f : \mathcal{U} \times \mathcal{I} \rightarrow \mathbb{R}$$

Predict relevance score $\rightarrow$ sort candidates $\rightarrow$ output top-K

Input Data

User ID + Context

user_id, learning_history, device, time, active_course_id...

Output Feed

Top-K Items

Sorted slate of concepts, lessons, or exercises matching user level

PART 1 · THE PROBLEM

The Information Overload Era

2.5 ZB

Daily Data Created

500M+

YouTube Catalog Size

100M+

Spotify Song Index

8.5B

Google Searches/day



User Paralysis

Faced with infinite choices, users encounter decision fatigue and quickly exit the application.



RS Remedy

Filters noise. Creates a unique portal tailored exactly to each individual user's needs.



Business Return

Increases session lengths, monthly active user retention, conversion rates, and lifetime value.

⚡ Special Application in EdTech

Students learn at **different paces**. Presenting content that is too easy triggers boredom; content that is too hard triggers dropout. RS balances the challenge curve.

Conversational Recommender Systems



Dialogue-Driven Personalization

Unlike static one-way recommendations, CRS **interactively queries** users to narrow down preferences, explain logic, and refine results in real-time.

1

Elicitation — Intent discovery

AI asks: "Do you want to practice quizzes or read theory?" to register explicit intents.

2

Recommendation — Slate delivery

Provides immediate items matching the user's conversational response.

3

Feedback Loop — Immediate correction

User replies: "This concept is too basic" $\rightarrow$ system shifts difficulty instantly.

4

Explanation — Building trust

"I am recommending this Python guide because you failed the Quick Check on loops."

STATIC RS VS. CONVERSATIONAL RS

ATTRIBUTE	STATIC RS	CONVERSATIONAL RS
Interaction	Passive consumption	Active conversation
Feedback Signal	Implicit clicks	Explicit dialogue + clicks
Cold Start	Suffers severely	Solves by asking
Explainability	Black-box score	Natural language dialogue
Adaptation Time	Batch offline (daily)	Real-time inside session
Complexity	Standard ML models	High (NLP + RL Agents)

PART 1 · CLASSIFICATIONS

Recommender System Taxonomy



Collaborative Filtering (CF)

"Users similar to you also liked..." Matches historical behaviors across populations.

User-based

Item-based

Matrix Factorization

⚠️ Vulnerable to Cold Starts & Data Sparsity



Content-Based Filtering

"You liked X, item Y is similar to X, hence you will like Y." Relies on metadata tags.

TF-IDF

Embeddings

Cosine Similarity

✅ Requires zero peer interaction logs



Hybrid Systems

Combines Collaborative, Content-Based, and Knowledge-Graph rules to eliminate single-model bottlenecks.

Weighted

Switching

Cascading



Knowledge-Based

Uses deterministic graph pathways and domain rules (e.g., prerequisite lesson hierarchies) to recommend.

Neo4j Graph

Graph Neural Nets

Prerequisites

Production Best Practice: Modern high-traffic recommender backends are almost exclusively **Cascaded Hybrids**.

PART 2 · DATA REPRESENTATIONS

Data Modeling for Recommenders

THE 3 DATA PILLARS

**User Vector**

user_id, demographic context, historic logs, mastery vectors, preferences...

**Item Vector**

item_id (lesson/node), course_id, difficulty levels, tag keywords, content embedding...

**Interaction Data**

Logs of user × item actions: action_type, timestamps, completion status, score...

USER-ITEM INTERACTION MATRIX

	node_1001	node_1002	node_1003
user_183	9	0	5
user_187	17	12	0
user_115	0	6	11

Implicit Affinity Score: Constructed by summing weighted interactions per user-concept pair (complete = 2.0, correct check = 2.0, view = 0.5...).

FEEDBACK SIGNALS

Implicit Feedback

Views, complete events, duration.
Abundant but noisy.

Explicit Feedback

Likes, star ratings, ratings. High
signal but scarce.

PART 2 · DATA BOTTLENECKS

Handling the Cold Start Problem



New User Cold Start

No behavioral history exists.
Collaborative filtering yields zero candidates.



New Item Cold Start

A new lesson is published. No student has clicked it yet, creating zero logs.



Sparse Interactions

Inactive users have minimal actions, causing high variance predictions.

✂ Mitigation Tactics

- | | | | |
|--|--|---------------------------------|--|
| ▶ Popularity fallback | — Serve top catalog items when user log size is zero | ▶ Demographic clustering | — Group users by profile fields (e.g., job role, level) |
| ▶ Interactive Elicitation | — AI asks onboarding style questions to build initial vector | ▶ Content Similarity | — Serve new items that resemble already popular lessons |
| ▶ Knowledge Graph Prerequisites | — Follow graph paths to recommend starting nodes | ▶ Synthetic Ingest | — Generate initial training interactions to pre-warm weights |

✓ **BDC Applied logic:** The Lakehouse view triggers a `learn_popular_lesson` fallback for cold users. AI preferences assign a heavy score weight of 3.0 to bootstrap CF matrices.

Recommender Feature Engineering

USER FEATURES TABLE

FEATURE NAME	DATATYPE	FUNCTIONAL PURPOSE
mastery_level	Float	Running score of concept mastery (0-1)
check_accuracy	Float	Ratio of correct quick checks
engagement_level	Enum	Activity tiers (Highly Active, Normal, At Risk)
learning_style	Enum	Behavioral styles (Practical, Theoretical)
struggle_nodes	Array	IDs of nodes flagged with error rate > threshold

ITEM FEATURES TABLE

FEATURE NAME	DATATYPE	FUNCTIONAL PURPOSE
difficulty	Float	Academic complexity rating (1-5)
topic_embedding	Vector	LLM embeddings representing content semantics
prerequisites	Array	Parent node IDs in the knowledge hierarchy
popularity_score	Float	Global interaction frequency rank

INTERACTION MATRIX SCORING LOGIC

```
# Explicit Interaction weights mapping
weights = {
    'lesson_complete': 2.0,
    'quick_check_correct': 2.0,
    'ask_ai': 1.5,
    'flashcard_flip': 1.0,
    'preference_elicited': 3.0, # CRS explicit statement
    'lesson_view': 0.5,
    'quick_check_wrong': 0.5,
}

affinity_score = sum(
    weights[action] for action in user_action_sequence
)
```

TEMPORAL DYNAMICS FEATURES

- ▶ **Exponential recency decay** — Recent actions represent current interests
- ▶ **Session density** — Interaction counts in active windows
- ▶ **Lapse interval** — Time elapsed since last login to capture churn

PART 2 · MASTERY MODELING

Bayesian Knowledge Tracing (BKT)

Why BKT?

Standard recommenders know *what you visited*. BKT models *whether you understood it*. It tracks the hidden state of concept mastery based on correct/incorrect response sequences.

BKT PROBABILITY EQUATIONS

$$P(L_{t-1} \mid \text{Correct}) = \frac{P(L_{t-1}) \cdot (1 - P(S))}{P(L_{t-1}) \cdot (1 - P(S)) + (1 - P(L_{t-1})) \cdot P(G)}$$

$$P(L_{t-1} \mid \text{Incorrect}) = \frac{P(L_{t-1}) \cdot P(S)}{P(L_{t-1}) \cdot P(S) + (1 - P(L_{t-1})) \cdot (1 - P(G))}$$

$$P(L_t) = P(L_{t-1} \mid \text{Action}) + (1 - P(L_{t-1} \mid \text{Action})) \cdot P(T)$$

STEP-BY-STEP BKT UPDATE MECHANICS

- 1 **Fetch Prior: $P(L_{t-1})$**
Initial probability of mastery (starts at $P(L_0)$ parameter).
- 2 **Assess action outcome (correct/incorrect)**
Student attempts a check. Correct counts map to probability updates.
- 3 **Compute Posterior $P(L_{t-1} \mid \text{Action})$**
Applies Bayes' theorem using slip $P(S)$ and guess $P(G)$ factors.
- 4 **Apply learning transition $P(T)$**
Accounts for learning event to yield final mastery state $P(L_t)$.

DOWNSTREAM RS APPLICATIONS

- ▶ **$P(\text{mastery}) < 0.8$** $\rightarrow$ Flags knowledge gap $\rightarrow$ RS boosts practice items
- ▶ **$P(\text{mastery}) \geq 0.8$** $\rightarrow$ Mastery reached $\rightarrow$ System unlocks next lesson node
- ▶ **Performance decline** $\rightarrow$ Triggers Spaced Repetition review

Collaborative Filtering Paradigms

Memory-Based CF

User-User CF Similarity

Finds peers with similar history using cosine similarity metrics.

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \times \|v\|}$$

Item-Item CF

Scores similarity between items directly. More stable in production since catalog changes slowly.

Model-Based CF (Matrix Factorization)

Alternating Least Squares (ALS) Step-by-Step

Decomposes interaction matrix by optimizing user and item factors alternatively.

- ▶ **Step 1:** Freeze V , solve U by minimizing squared reconstruction error.
- ▶ **Step 2:** Freeze U , solve V by minimizing squared reconstruction error.
- ▶ **Step 3:** Repeat until convergence metric is satisfied.

MATRIX DECOMPOSITION FORMULATION

$$R \approx U \times V^T$$

```
# ALS Factorization Epoch Loop
for epoch in range(num_epochs):
    # Solve U with V fixed:
    # U = R @ V @ (V.T @ V + lambda * I)^-1

    # Solve V with U fixed:
    # V = R.T @ U @ (U.T @ U + lambda * I)^-1
```

TRADE-OFF ANALYSIS

METHOD	PRECISION	SCALABILITY	NEW USER COLD START
User-User	Medium	Low ❌	Poor ❌
Item-Item	Medium	Medium ⚡	Poor ❌
ALS (implicit)	High	High ✅	Poor ❌
Neural CF	Very High	High ✅	Moderate ⚡

PART 3 · ADVANCED ALGORITHMS

Sequential Recommendation

Why Sequential Models?

Learning order is strict: A $\rightarrow$ B $\rightarrow$ C differs from B $\rightarrow$ A $\rightarrow$ C. Sequential recommender models treat user history as an ordered sequence of events to predict the next action.

GRU4REC FORWARD PATH STEP-BY-STEP

- 1 Represent history sequence**
Input sequence of item IDs: $[i_1, i_2, \dots, i_{t-1}]$.
- 2 Project to Embedding Space**
Embeds each item ID into a dense vector representation.
- 3 Process with recurrent cells**
Feeds embedded vectors into GRU cells to accumulate sequential dependencies.
- 4 Map hidden state to logits**
Decodes last hidden vector h_{t-1} to calculate next item vocabulary logits.

```
class GRU4RecBackbone(nn.Module):
    def __init__(self, vocab_size, hidden=128):
        super().__init__()
        self.item_emb = nn.Embedding(vocab_size, 64)
        self.gru = nn.GRU(64, hidden, num_layers=2)
        self.fc = nn.Linear(hidden, vocab_size)

    def forward(self, seq):
        x = self.item_emb(seq)
        h, _ = self.gru(x)
        logits = self.fc(h[:, -1, :])
        return logits
```

SRD COMPOSITE LOSS EQUATION

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{diversity}}$$

Reinforcement Learning **Fine-Tuning**

RL Tuning Paradigm

Pre-train sequential models offline. Then, treat recommendation slates as actions and update the policy using **REINFORCE** gradient updates based on user success rewards.

REINFORCE POLICY UPDATE STEP-BY-STEP

- 1 Run policy forward pass**
Generate model logits $\pi_{\theta}(a|s)$ representing probability values over catalog items.
- 2 Sample K actions**
Sample a slate of Top-K recommendation items using multinomial sampling.
- 3 Acquire environmental reward R**
Measure user engagement: $\text{reward} = \text{nDCG@K} + 0.2 \times \text{item novelty}$.
- 4 Compute loss & update weights**
Update network weights using gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a|s)$.

POLICY GRADIENT (REINFORCE) EQUATION

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a | s) \cdot R(a, s)]$$

```
def train_rl_step(model, ref_model, batch):  
    logits = model(batch['seq'])  
    actions = sample_top_k(logits, k=5)  
    rewards = compute_rewards(actions, batch)  
  
    # KL Divergence constraint (DivKG)  
    kl = kl_div(logits, ref_model(batch))  
    loss = -(rewards * logits).mean() + 0.05 * kl  
  
    loss.backward(); optimizer.step()
```

PART 3 · MEMORY RETENTION

Spaced Repetition Systems (SRS)



Ebbinghaus Forgetting Curve

Memory retention decays exponentially. Spaced repetition calculates the **optimal review date** — right before the user is likely to forget — to lock knowledge in long-term memory.

SM-2 SPACING FORMULAS

$$EF' = EF + (0.1 - (5 - q) \cdot (0.08 + (5 - q) \cdot 0.02))$$

$$I(n) = I(n - 1) \cdot EF$$

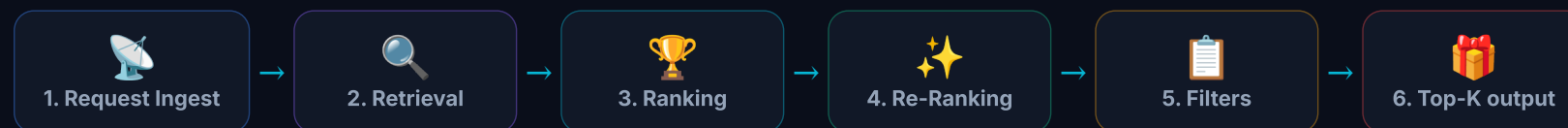
SUPERMEMO-2 ALGORITHM STEP-BY-STEP

- 1 Collect quality score (0-5)**
User rates recall performance on a flashcard.
- 2 Adjust Easiness Factor (EF)**
Update easiness factor based on rating value.
- 3 Calculate next interval (I)**
Calculates spacing interval in days for next repetition.
- 4 Schedule next review date**
Set next_review_date to current time + interval days.

```
-- flashcard_repetitions schema
CREATE TABLE flashcard_repetitions (
  flashcard_id BIGINT,
  student_id BIGINT,
  easiness_factor FLOAT DEFAULT 2.5,
  interval INT DEFAULT 1,
  repetition_count INT DEFAULT 0,
  next_review_date TIMESTAMP
);
```

PART 3 · ARCHITECTURE

End-to-End Serving Pipeline

**1 Candidate Generation**

Vector search (ANN) + graph prerequisites extraction. Selects ~500 candidates out of 100K+ items in < 10ms.

2 Ranking Score

GRU4Rec model outputs sequence logits. Combines embedding similarity and mastery probabilities to assign score weights.

3 Real-Time Re-Ranking

Applies policy gradient networks (DivKG) to rearrange slots, maximizing diversity metrics and novelty.

4 Hard Filters

Strips out already-completed items. Ensures prerequisite nodes are met. Drops out-of-course concepts.

5 Context Boost

Forces SRS due cards to priority positions. Elevates active struggle nodes to catch students before they slip.

6 Delivery & Log

Serves the top-K array. Registers impression events to Kafka to close the system feedback loop.

PART 4 · EVALUATION

Offline Evaluation Metrics

Accuracy Equations

$$\text{Precision@}K = \frac{|\text{Relevant} \cap \text{Recommended}|}{K}$$

$$\text{Recall@}K = \frac{|\text{Relevant} \cap \text{Recommended}|}{|\text{Relevant}|}$$

$$\text{nDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}$$

Beyond-Accuracy Metrics

ILD (Diversity)

Intra-List Distance. Higher values mean more diverse lists.

Novelty

$-\log_2(\text{popularity})$. Measures how unexpected recommendations are.

Catalog Coverage

% of distinct items recommended to at least one user.

Serendipity

Surprising yet highly relevant recommendations.

Offline Model Benchmarks

MODEL PIPELINE	P@5	RECALL@5	NDCG@5	ILD
Baseline Heuristic	0.40	0.25	0.38	0.72
Cosine Similarity CF	0.48	0.31	0.45	0.65
ALS Latent Factors	0.55	0.38	0.52	0.68
Sequential GRU4Rec	0.63	0.45	0.61	0.70
Production SRD + DivKG RL	0.71	0.52	0.69	0.81

⚠ Offline Warning: Excellent offline metrics do not guarantee live user conversion. Live online A/B testing is mandatory.

PART 4 · EVALUATION

Online A/B Testing

A/B Testing Framework

Directs incoming traffic randomly to Control (model A) or Treatment (model B). Evaluates real business changes over a set period.

- 1 Hypothesis Design**
Defines target variables, target metrics, and minimal sample size requirements (power analysis).
- 2 Hash-Based Traffic Split**
Computes $\text{Hash}(\text{user_id} + \text{salt}) \% 100$ to enforce static cohort mapping.
- 3 Run & Monitor (10-14 days)**
Collects metrics. Controls for temporal novelty effects. Tracks latency.
- 4 Statistical Test**
Performs t-tests or z-tests. Validates that $p\text{-value} < 0.05$ before shipping the new model.

KEY ONLINE METRICS

Click-Through Rate (CTR)

Clicks / Impressions of recommended cards

Lesson Completion Rate

Ratio of recommended lessons completed

Active Session Duration

Mean time spent learning per user session

Retention (D7 / D30)

% of users returning after N days

Guardrail Metrics: Keep an eye on error rates and latency. If p95 latency spikes above 200ms or error rates exceed 1%, abort the experiment immediately.

PART 4 · VALIDATION DESIGN

Quy trình Temporal Evaluation

⚠ Preventing Data Leakage

Never split recommendation datasets randomly. Because learning behavior is chronological, random splits leak future interactions. Always partition using a **Temporal Split**.

TEMPORAL SPLIT PROTOCOL STEP-BY-STEP

- 1 Sort user interactions chronologically**
Order all logged student actions by their timestamp field.
- 2 Establish division boundary**
Define a boundary cut point (e.g. last 20% of actions per user).
- 3 Train on past logs**
Train recommendation algorithms exclusively on pre-split historical data.
- 4 Evaluate predictions on future events**
Test predictions against post-split records (guarantees zero look-ahead bias).

IMPLEMENTATION CODE PATTERN

```
def temporal_split(df, test_ratio=0.2):  
    # Sort chronologically  
    df = df.sort_values('timestamp')  
    # Slice the last 20% records per user  
    test_idx = df.groupby('user_id').apply(  
        lambda g: g.tail(int(len(g) * test_ratio))  
    ).index.get_level_values(1)  
  
    train_set = df.drop(test_idx)  
    test_set = df.loc[test_idx]  
    return train_set, test_set
```

 Evaluated in: 08_offline_evaluation.ipynb

PART 5 · SYSTEMS ENGINEERING

Production Serving Architecture

The Multi-Stage Serving Flow

- 1 Retrieval Stage (< 10ms)**
FAISS vector lookups + Graph DB queries. Narrows down items from 100K+ to ~500 candidates.
- 2 Ranking Stage (< 50ms)**
Scores candidate items using GRU4Rec. Gathers user profiles from Feature Stores. Returns top-50.
- 3 Re-Ranking Stage (< 20ms)**
Applies business rules, skips completed items, checks graph prerequisites, and pushes SRS due cards.
- 4 Auditing & Logging**
Delivers recommendations. Dispatches impression payloads to Kafka to close the feedback loop.

INFRASTRUCTURE COMPONENTS

**Feature Store (Redis)**

Holds pre-computed user profiles and latent item embeddings. Serving latency P99 < 2ms.

**Vector Database (Qdrant)**

Conducts fast Approximate Nearest Neighbor searches over embedding spaces.

**Model Registry (MLflow)**

Tracks model versions, parameters, and metadata. Enables 1-click rollbacks.

**Telemetry (Prometheus + Grafana)**

Monitors serving latency, throughput, prediction drift, and error codes.

PART 5 · AUTOMATION

ML Training Pipelines

Scheduled Pipeline Step-by-Step

- 1 Run hourly Lakehouse Parquet Archiving**
Background cron `archive\_scheduler` moves events from Postgres to Bronze Parquet folders.
- 2 Execute silver-to-gold ETL transformations**
DuckDB queries aggregate raw events into user affinity scores and student course metrics.
- 3 Train sequential model weights (weekly)**
Triggers GPU training scripts over unified historical Parquet logs.
- 4 Deploy model to staging registry**
Saves model binaries to MLflow registries and initiates canary splits.

ONLINE API INFERENCE ENDPOINT

```
@router.get("/personalize/student/{user_id}")
async def serve_recommendations(user_id: int, course_id: int):
    # 1. Fetch user mastery profile from DuckDB
    profile = lakehouse.get_student_profile(user_id, course_id)

    # 2. Run GRU4Rec network over history sequence
    recs = model.predict(profile['history'])

    # 3. Contextual boost using due SRS items
    final_slate = rerank(recs, profile['srs_due'])
    return final_slate[:5]
```

PRODUCTION SLA TARGETS

< 100ms

P99 Serving Latency

99.95%

API Availability

PART 5 · MODEL OPERATIONS

Recommender MLOps Lifecycle



Drift Detection

Tracks changes in user action distributions. Automatically triggers retraining when feature drift exceeds thresholds.



Online Tracking

Monitors click-through rates and lesson completion rates. Alerts when engagement drops by > 5%.



Fail-Safe Fallbacks

If the neural ranking network times out, the API automatically falls back to popular items in under 5ms.

The Feedback Loop Architecture

Student Clicks	→ Kafka Ingestion
Personalize Service	→ DuckDB Lakehouse
Retrain Pipeline	→ Shadow / Canary Rollout

SAFE DEPLOYMENT CHECKLIST

- ▶ **Verification** — Ensure offline nDCG@5 score exceeds previous production metrics
- ▶ **Shadow Testing** — Run the new model silently in parallel to inspect outputs
- ▶ **Canary Rollouts** — Route traffic gradually: 5% $\rightarrow$ 20% $\rightarrow$ 100%
- ▶ **Guardrail Limits** — Monitor error counts and latency SLAs closely
- ▶ **Automatic Rollback** — Revert to the stable version if error rates exceed 1%

PART 6 · BDC INTEGRATION

BDC System Architecture

MICROSERVICES BLUEPRINT

**lms-service (Go)**

Handles user interaction clicks and writes to Postgres; publishes events to Kafka.

**personalize-service (Python)**

Consumes Kafka logs, updates DuckDB views, and serves the recommendation API.

**ai-service (FastAPI + LLM)**

Runs the AI Mentor. Invokes `get_study_plan` tool to fetch user profiles.

**frontend (Next.js)**

Renders the learning dashboard, quiz modules, and the AI chatbot panel.

**da-analytics (Python workspace)**

Offline training sandbox for BKT, GRU4Rec, and DivKG models.

MEDALLION LAKEHOUSE LAYOUT (DUCKDB)

**Bronze Layer**

`bronze_interactions` — Raw JSON logs consumed from Kafka topics.

**Silver Layer**

`unified_interactions` — Cleaned, schema-conformed, unified view.

**Gold Layer — Business views**`user_item_matrix``concept_struggles``student_metrics``struggle_alerts``study_recommendations`

Real-Time: Kafka ingest to DuckDB updates take < 1s



Batch: Parquet exports run hourly via background crons

PART 6 · BDC STATUS

BDC's Current Recommender State

Active Production Heuristic RS

```
-- gold_study_recommendations logic
SELECT
  user_id,
  course_id,
  CASE
    WHEN has_struggle_concept THEN 'review_struggle_concept'
    WHEN check_accuracy < 0.60 THEN 'discuss_with_ai'
    WHEN viewed_count = 0 THEN 'learn_popular_lesson'
    ELSE 'learn_next_lesson'
  END AS recommended_action_type
FROM student_metrics;
```

AI Mentor Conversational Features

- ▶ **get_study_plan** — Merges Lakehouse metrics with SRS queues
- ▶ **Stuck detection** — Triggers alerts when struggle thresholds are crossed
- ▶ **Conversational onboarding** — Maps intents directly to interest vectors
- ▶ **Proactive notification** — Pushes reviews via Kafka when inactive

Future Upgrade Roadmap

- ✓ **Phase 1 · Heuristic + CRS Agent**
SQL heuristics + AI Mentor tools + Lakehouse pipelines are fully deployed.
- 2 **Phase 2 · Collaborative Filtering**
Train ALS and cosine similarity models over Gold user-item matrix files.
- 3 **Phase 3 · Sequential GRU4Rec**
Deploy sequential models using SRD loss to capture study patterns.
- 4 **Phase 4 · Policy Gradient tuning**
Apply REINFORCE loops to optimize live retention; launch A/B tests.

PART 6 · DATA ANALYST WORKFLOWS

DA Analytics Workspace

The 8 Interactive Notebooks

01 Data Exploration

Profiles the Gold Parquet layers and generates plots.

02 Heuristic Baseline

Calculates offline metrics for the active SQL strategy.

03 CF (Cosine & ALS)

Develops matrix factorizations and sparse CF indexes.

04 BKT Model

Fits BKT parameters to diagnose hard concepts.

05 Behavioral LSTM

Models session vectors to predict user friction scores.

06 SRD GRU4Rec

Trains GRU networks with Soft-Rank Diversity losses.

07 DivKG RL

Fine-tunes the policy network using REINFORCE.

08 Offline Comparison

Builds the master benchmark table comparing all models.

DA Quick Start Guide

```
# 1. Spin up the environment
cd da-analytics
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 2. Bootstrap data using the generator
python scripts/generate_mock_data.py --output-dir data/lakehouse/gold

# 3. Launch the Jupyter lab server
jupyter lab
```

Key Scripts Reference

- ▶ **generate_mock_data.py** — Builds synthetic databases for 100 users
- ▶ **train_srd.py** — Trains the sequential GRU4Rec network
- ▶ **train_bkt.py** — Computes mastery and fits BKT parameters
- ▶ **export_results.py** — Saves recommendation slates to CSV files
- ▶ **metrics.py** — Core evaluations (Precision, Recall, nDCG, ILD, Novelty)

SUMMARY

Summary & Next Steps

✓ Key Takeaways

- ▶ **RS** helps solve information overload by matching users with optimal content feeds.
- ▶ **CRS** turns recommendations into conversational exchanges, solving cold starts.
- ▶ **User modeling** combines implicit logs, BKT mastery estimates, and SRS review cards.
- ▶ **Algorithm evolution** goes from simple heuristics to CF, sequential GRU networks, and RL.
- ▶ **Validations** rely on offline evaluations and live online A/B testing splits.
- ▶ **Deployments** use a multi-stage flow (Retrieval $\rightarrow$ Ranking $\rightarrow$ Re-ranking).

🔄 Action Items

- 1 Run the 8 Analyst Notebooks**
Inspect training outputs and generate the master offline comparison benchmarks.
- 2 Accumulate Live Student Interaction Logs**
Needs >500 active users with >10 logs each to transition from heuristics to neural CF.
- 3 Execute GRU4Rec (Phase 3)**
Run training scripts locally: `python scripts/train_srd.py --epochs 20`.
- 4 Trigger A/B Tests on Personalize Service**
Deploy the GRU candidate generator alongside the active heuristic controller.



Additional Reading:

[Matrix Factorization \(Netflix, 2009\)](#)
[GRU4Rec \(Hidasi et al., 2016\)](#)
[Duolingo Spaced Repetition \(2016\)](#)
[CRS Survey \(Gao et al., 2021\)](#)